

EARN

Eliciting Actionable Recommendation Feedback

Codebase, database ও system architecture-এর সম্পূর্ণ টেকনিক্যাল ডকুমেন্টেশন

প্রজেক্ট: EARN একটি PhD study platform যা তিনটি ভিন্ন elicitation condition-এ newsletter feedback সংগ্রহ করে এবং actionability rubric দিয়ে score করে। Stack: Django REST Framework, Next.js, PostgreSQL, Docker Compose, self-hosted LLM।

রিপোজিটরি: `/data/munna/phd_project/earn`

Contents

1	ভূমিকা	2
1.1	EARN আসলে কী	2
1.2	Stimulus: POPROX-style newsletter	2
1.3	এই ডকুমেন্টের উদ্দেশ্য	2
2	System Architecture	3
2.1	Tech Stack	3
2.2	System Diagram	3
2.3	Design Rationale	4
3	Repository ও Code Structure	5
3.1	Django App কেন তিনভাগে	5
4	Database Design	6
4.1	Overview	6
4.2	Design Principles	6
4.3	Field Reference	7
4.3.1	Newsletter	7
4.3.2	Participant	7
4.3.3	FeedbackResponse (Participant-এর সাথে 1:1)	7
4.3.4	SurveyResponse (Participant-এর সাথে 1:1)	8
4.3.5	ActionabilityRating (FeedbackResponse-এর সাথে 1:N)	8
4.4	Participant Status ও study_phase	9
4.4.1	status field	9
4.4.2	study_phase field	9
5	Backend: App Structure, Authentication ও Permissions	10
5.1	Settings ও Configuration	10
5.2	Authentication: JWT	10
5.3	Authorization: দুই ধরনের researcher role	10
6	Balanced Random Assignment	11
7	Participant Flow	12
7.1	ধাপে ধাপে যাত্রা	12
8	তিনটি Feedback-Elicitation Condition	13
9	Condition 3: Interactive Feedback Assistant	14

9.1	Strict Rules	14
9.2	System Prompt কোথায় থাকে	14
9.3	Conversation Flow	15
9.4	Newsletter Grounding (read-only context)	15
9.5	Local-only Model Client — কোনো Fallback নেই	15
9.6	Final Consolidation (final_draft বনাম final_text)	16
9.7	Actionability Rubric	16
10	Researcher Dashboard	17
11	Statistical Analysis	18
12	API Reference	19
13	Frontend Architecture	21
13.1	Folder Organization	21
13.2	Key Component: NewsletterView	21
13.3	Key Component: FeedbackStep	21
13.4	API Client: JWT Auto-refresh	22
14	Deployment ও Operations	23
14.1	Docker Compose Services	23
14.2	run.sh — Convenience Wrapper Script	23
14.3	Environment Variables	23
14.4	Port Map	24
15	Testing ও Quality	25
16	Security ও Privacy Considerations	26
17	সম্ভাব্য প্রশ্ন ও উত্তর	27
18	Glossary	29

1 ভূমিকা

1.1 EARN আসলে কী

EARN (Eliciting Actionable Recommendation feedback from users for News personalization) একটি web-based research platform যা একটি PhD study চালানোর জন্য build করা হয়েছে। Core research question হলো: একজন reader যখন একটি personalized news newsletter পড়ে open-ended feedback দেন, কোন elicitation method reader-কে বেশি actionable feedback দিতে সাহায্য করে — অর্থাৎ, এমন feedback যা একটি recommender system বাস্তবে কাজে লাগাতে পারে।

এর জন্য একটি between-subjects experiment design করা হয়েছে — প্রতিটি participant randomly তিনটি condition-এর একটির মধ্যে পড়েন:

1. Just Ask — শুধু একটি neutral prompt, কোনো guidance ছাড়াই।
2. Examples + Instructions — prompt-এর সাথে একটি explanation, কিছু guiding question এবং একটি example।
3. Interactive Feedback Assistant — একটি LLM-চালিত assistant-এর সাথে ছোট conversation, যা reader-কে তাঁর নিজের feedback clarify করতে সাহায্য করে (কিন্তু কখনো তাঁর হয়ে লিখে দেয় না)।

পরে researcher-রা প্রতিটি final feedback-কে একটি ৫-dimension actionability rubric দিয়ে score করেন — blind multi-rater dashboard-এর মাধ্যমে একাধিক human rater স্বাধীনভাবে score করেন (কোনো LLM auto-rating এখন নেই)। Statistical analysis এই application-এর অংশ নয়; CSV export থেকে R/Python-এ আলাদাভাবে করা হয়।

এই platform feedback সংগ্রহ ও পরিমাপ করে — এটি কোনো live newsletter system-এ feedback apply করে content পরিবর্তন করে না (README-এ এটি স্পষ্টভাবে future work বলে চিহ্নিত)। Newsletter-গুলো সম্পূর্ণ static/fixed stimulus, প্রতিটি participant পড়ার আগে থেকেই তৈরি।

1.2 Stimulus: POPROX-style newsletter

Participant-রা যে newsletter পড়েন তা real Associated Press (AP) article দিয়ে তৈরি, এবং POPROX reference template-এর (https://github.com/Mahamudul42/popprox_newsletter) visual style follow করে (teal masthead, dark-blue section bar, thumbnail + topic-label + serif headline + summary)। প্রতিটি newsletter-এর format fixed: ৫টি section, প্রতি section-এ ৩টি article। তিনটি edition বিভিন্ন topic-এ emphasis দেয় (world, US/politics, tech/sports), যাতে feedback-এ diversity আসে। Data থাকে backend/apps/study/seed_data/newsletters.json-এ, seed_study management command দিয়ে database-এ load হয়।

1.3 এই ডকুমেন্টের উদ্দেশ্য

প্রজেক্ট owner হিসেবে পুরো system-টি — architecture, data model, প্রতিটি design decision-এর কারণ — ভালোভাবে বোঝা দরকার। এই ডকুমেন্ট সেই লক্ষ্যে: architecture থেকে শুরু করে প্রতিটি database table, API endpoint, frontend-এর গুরুত্বপূর্ণ component, এবং শেষে সম্ভাব্য প্রশ্নোত্তর নিয়ে আলোচনা করা হয়েছে। প্রতিটি তথ্য সরাসরি কোড পড়ে verify করা — কোনো অংশ অনুমান করে লেখা হয়নি।

2 System Architecture

2.1 Tech Stack

Layer	Technology	Role
Frontend	Next.js 16 (App Router), React 18, TypeScript, Tailwind CSS, lucide-react, Vitest	Participant UI + researcher dashboard
Backend	Django 5, Django REST Framework, SimpleJWT, drf-spectacular	REST API, business logic, authentication
Database	PostgreSQL 16	সমস্ত persistent data
LLM	Self-hosted OpenAI-compatible endpoint (default model openai/gpt-oss-120b)	Condition-3 assistant-এর জন্য
Container	Docker, Docker Compose	৩-service orchestration (§ 14)

2.2 System Diagram

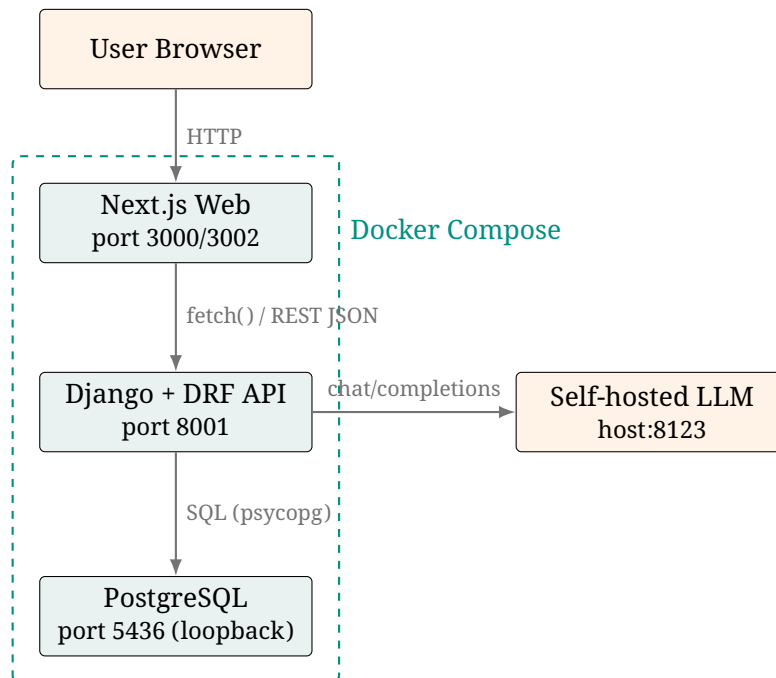


Figure 1: EARN-এর high-level architecture। Docker Compose তিনটি service চালায় (db, backend, web); self-hosted LLM host machine-এর একটি আলাদা process (compose-এর বাইরে)।

2.3 Design Rationale

- ৩টি Docker service (db, backend, web) — `docker-compose.yml`-এ defined। কোনো আলাদা background worker নেই — Condition-3-এর LLM call participant-এর নিজের request-এর মধ্যেই synchronously হয়, এবং statistical analysis এই application-এর বাইরে (CSV export-এর পর R/Python-এ) হয়, তাই কোনো job queue দরকার পড়েনি।
- **network_mode: host** backend-এ ব্যবহৃত হয়েছে যাতে container host machine-এর 127.0.0.1:8123-এ চলা LLM-এ সরাসরি পৌঁছাতে পারে — এই server-এ Docker-এর bridge-network hairpin NAT সমস্যার কারণে এটি প্রয়োজন হয়েছিল।
- PostgreSQL, SQLite নয় — production settings (`settings/prod.py`) নিশ্চিত করে যে DATABASE_URL সবসময় PostgreSQL-এর দিকে point করে; এটি concurrent write, JSON field, এবং heavy statistical read-query-র জন্য বেশি উপযুক্ত।

3 Repository ও Code Structure

Path	Content
backend/apps/core/	Health check, pagination, permission class (কোনো model নেই)
backend/apps/users/	JWT authentication, researcher/rater role
backend/apps/study/	Core app — model, view, serializer, assignment logic, LLM assistant, management command
backend/project_backend/settings/	base.py, dev.py, prod.py — environment-driven config
backend/system_prompt.txt	Condition-3 assistant-এর live system prompt (advisor-supplied, POPROX-specific)
backend/tests/	pytest test suite
web/src/app/	Next.js App Router page (participant + researcher)
web/src/components/	shell/, study/, ui/ — reusable React component
web/src/lib/	api.ts (API client), auth.ts (token), types.ts (TypeScript type)
IRB/	IRB (Institutional Review Board) protocol document
docker-compose.yml, run.sh	Orchestration ও operations script

3.1 Django App কেন তিনভাগে

Django convention অনুযায়ী প্রতিটি “app” একটি self-contained responsibility বহন করে। এখানে core (generic infrastructure), users (identity/access), এবং study (domain logic) আলাদা রাখা হয়েছে যাতে authentication/permission code business logic থেকে আলাদা থাকে — এটি test করা ও পরিবর্তন করা সহজ করে।

4 Database Design

4.1 Overview

Study app-এ ৫টি custom model আছে, এছাড়া Django-র built-in User/Group table ব্যবহার করা হয়েছে researcher ও rater account-এর জন্য (আলাদা কোনো Profile model নেই)। Migration history: 0001_initial থেকে 0011-পর্যন্ত — ১১টি migration। প্রথম আটটি ধাপে ধাপে feature যোগ করেছে (compact survey, chat log, final draft, LLM-rating/analysis স্কিমা, unique rating constraint); পরের তিনটি (0009–0011) সেই LLM-rating ও analysis-সংক্রান্ত table/column বাদ দিয়েছে এবং chat_log/edition_label-কে পরিষ্কার করেছে, যখন সেই feature-গুলো codebase থেকে সরিয়ে ফেলা হয়েছিল।

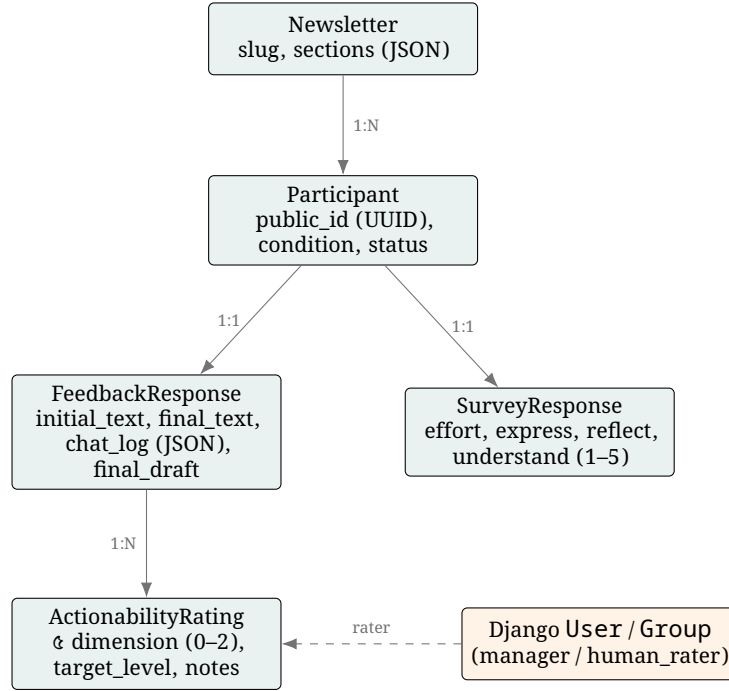


Figure 2: EARN database-এর entity-relationship (ER) diagram। Dashed arrow Django User-এর সাথে ActionabilityRating.rater relationship নির্দেশ করে (rater account delete হলে SET_NULL)।

4.2 Design Principles

- **public_id (UUID)** প্রতিটি Participant-এর primary key id থেকে আলাদা — এটি guessable নয়, তাই URL-এ ব্যবহার করা safe, এবং এটিই participant-এর completion code হিসেবে দেখানো হয় (Prolific-compatible)।
- **JSONField দুই জায়গায়:** Newsletter.sections (nested section/article structure) এবং FeedbackResponse.chat_log (পুরো conversation)। এদের structure বারবার বদলায় না বলে আলাদা table না বানিয়ে JSON-এ রাখা সহজ ও যথেষ্ট।
- **on_delete policy সচেতনভাবে বাছাই করা:** Newsletter delete করা যাবে না যদি কোনো Participant সেটি ব্যবহার করে থাকে (PROTECT); কিন্তু কোনো rater account delete হলেও তাঁর দেওয়া rating টিকে থাকে, শুধু rater field NULL হয়ে যায় (SET_NULL) — research data কখনো হারায় না।
- **total score store করা হয় না** — ActionabilityRating.total একটি Python @property (৫টি dimension-এর sum), database-এ আলাদা column নেই, তাই কখনো inconsistent (stale) হতে পারে না।

- একটি unique constraint: (feedback, rater) — একজন rater একই response দ্বিতীয়বার rate করতে পারবেন না (RatingCreateView duplicate চেষ্টায় HTTP 409 দেয়)।

4.3 Field Reference

4.3.1 Newsletter

Field	Type	Description
slug	Slug, unique	URL-friendly identifier (যেমন edition-a)
title	CharField(160)	Newsletter-এর title
sections	JSON	[{name, articles: [{label, headline, summary, image, url}]}] — fixed ৫×৩ format
is_active	Boolean	Inactive করলে নতুন assignment-এ ব্যবহার হবে না
created_at	DateTime	auto_now_add

edition_label (যেমন “Saturday Edition · June 21, 2026”) store করা হয় না — আগে একটি stored column ছিল, migration ০০১১ সেটি বাদ দিয়েছে; এখন এটি content.current_edition_label() দিয়ে request-time-এ আজকের তারিখ থেকে compute হয় (SerializerMethodField), যাতে stimulus কখনো পুরনো তারিখ দেখায় না।

4.3.2 Participant

Field	Type	Description
public_id	UUID, unique	Participant-এর URL, completion code হিসেবেও ব্যবহৃত
condition	Integer (1/2/3)	Just Ask / Examples / Assistant
newsletter	FK Newsletter (PROTECT)	→ কোন newsletter দেখানো হয়েছে
recruitment_source	Choice	direct, movielens, prolific, other
external_ref	CharField(120)	যেমন Prolific PID (?ref=)
status	Choice	নিচে § 4.4.1 দেখুন
consented_at, completed_at	DateTime, nullable	
study_phase	Choice	preview, pilot, main (§ 4.4.2)
created_at	DateTime	

4.3.3 FeedbackResponse (Participant-এর সাথে 1:1)

Field	Type	Description
initial_text	Text	প্রথম যা লিখেছেন (Condition 1/2-এ final_text-এর সমান)
final_text	Text	যা submit হয়েছে (analysis-এর মূল data)
assistant_action	Choice	none, suggestion, question, ok — assistant-এর সর্বশেষ action
assistant_message	Text	Assistant-এর সর্বশেষ message
chat_log	JSON list	পুরো Condition-3 conversation: {role, content, action?, ts}
final_draft	Text	LLM-consolidated draft (§9) — final_text থেকে আলাদা রাখা হয় যাতে participant কতটা edit করলেন তা measure করা যায়
revision_count	PositiveInt	Condition 3-এ follow-up message-এর সংখ্যা
time_on_task_seconds	PositiveInt, nullable	
created_at, updated_at	DateTime	

4.3.4 SurveyResponse (Participant-এর সাথে 1:1)

চারটি field, প্রতিটি ১-৫ scale-এ (Likert/scale-estimation): effort (cost), express (self-efficacy), reflect (seriousness check), understand (perceived actionability)। প্রতিটি ভিন্ন construct measure করে (RQ3, §11)।

4.3.5 ActionabilityRating (FeedbackResponse-এর সাথে 1:N)

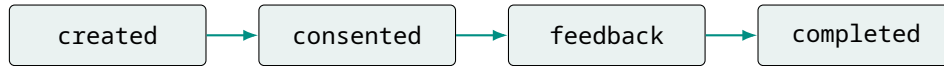
Field	Type	Description
rater	FK → User, nullable	যে human rater score দিয়েছেন (rater account delete হলে NULL)
target_specificity	প্রতিটি 0/1/2	Rubric-এর ৫টি dimension (§9.7)
direction_operation		
collection_allocation		
context_persistence		
system_feasibility		
target_level	Choice	article, section, collection, unclear
notes	Text	

total (0–10) store করা হয় না — এটি উপরের ৫টি dimension-এর sum হিসেবে runtime-এ compute হয়।

4.4 Participant Status ও study_phase

4.4.1 status field

Model-এ ৪টি value defined: created, consented, feedback, completed (apps/study/models.py) — একটি linear progression, views.py-র প্রতিটি step এর মধ্যে একটিকে set করে:



4.4.2 study_phase field

Value	Meaning
preview	?condition= বা ?newsletter= দিয়ে জোর করে খোলা session (demo/preview)। Automatically set হয়।
pilot	Default — STUDY_PHASE environment variable set না করলে সব সাধারণ participant এই phase-এ পড়েন।
main	Real recruitment-এর আগে STUDY_PHASE=main set করে stack restart করতে হয়।

কেন দরকার: demo/test session যেন confirmatory analysis-এ ঢুকে না পড়ে তা নিশ্চিত করতে। Balanced assignment (§ 6) এবং overview dashboard — দুটোই phase অনুযায়ী আলাদাভাবে count করে।

5 Backend: App Structure, Authentication ও Permissions

5.1 Settings ও Configuration

project_backend/settings/ তিনভাগে বিভক্ত: base.py (common config), dev.py (DEBUG=True, CORS_ALLOW_ALL_ORIGINS=True), prod.py (strict CORS/CSRF allowlist)। দুটোই PostgreSQL ব্যবহার করে — SQLite কোথাও নেই। django-environ library দিয়ে প্রতিটি setting environment variable থেকে আসে (root .env file থেকে পড়া হয়) — কোনো secret সরাসরি code-এ লেখা নেই।

মূল settings: JWT (SIMPLE_JWT: access token ৮ ঘণ্টা, refresh token ৭ দিন), DRF pagination (page-এ ২৫টি, DefaultPagination), drf-spectacular দিয়ে automatic OpenAPI documentation (/api/docs/)।

5.2 Authentication: JWT

Researcher/rater login করেন /api/auth/token/-এ username-password দিয়ে; বদলে পান access ও refresh token (SimpleJWT)। Frontend দুটোই localStorage-এ রাখে (lib/auth.ts)। প্রতিটি API call-এ Authorization: Bearer <access> header যায়; 4০1 পেলে lib/api.ts-এর request() function automatically refresh token দিয়ে নতুন access token নিয়ে আবার চেষ্টা করে (একবারই, infinite loop এড়াতে retried flag দিয়ে) — fail হলে token মুছে login page-এ পাঠিয়ে দেয়।

Participant-দের কোনো account লাগে না — public_id (UUID)-ই তাঁদের একমাত্র identity, যা login ছাড়াই কাজ করে (permission_classes = [AllowAny])।

5.3 Authorization: দুই ধরনের researcher role

Staff user (is_staff=True) দুইভাগে বিভক্ত, Django Group দিয়ে — apps/users/roles.py-এর HUMAN_RATER_GROUP constant-এ group-এর নাম human_raters defined:

Permission Class	Condition	Usage
IsResearcher	is_staff=True (যেকোনো staff)	Response list দেখা, rating submit করা
IsStudyManager	is_staff=True এবং human_raters group-এ নেই	Overview, CSV export, rater account management

অর্থাৎ human rater একটি limited role — তাঁরা শুধু blind rating queue দেখতে পারেন; study manager (মূল researcher account) সব দেখতে পারেন। Frontend-এ ResearcherShell component /api/auth/me/ call করে role জেনে navigation menu সীমিত করে এবং rater-কে জোর করে /researcher/rate-এ পাঠিয়ে দেয় (§ 10)।

কেন blind rating জরুরি: human rater-রা কখনো দেখেন না কোন condition-এ feedback এসেছে, LLM কী score দিয়েছে, বা অন্য কোনো human rater কী score দিয়েছেন (BlindFeedbackSerializer শুধু final_text পাঠায়)। এটি anchoring bias এড়ানোর জন্য।

6 Balanced Random Assignment

প্রতিটি নতুন participant-কে (condition, newsletter)-এর একটি cell-এ বসানো হয় — ৩টি condition $\times$ ৩টি newsletter = ৯টি সম্ভাব্য cell। Logic আছে apps/study/assignment.py-এর choose_cell() function-এ:

1. বর্তমান study_phase-এ (§ 4.4.2) প্রতিটি cell-এ এখন পর্যন্ত কতজন আছেন তা count করা হয়।
2. যে cell(গুলো)-তে সংখ্যা সবচেয়ে কম, তাদের মধ্যে থেকে randomly একটি বেছে নেওয়া হয় (tie ভাঙার জন্য random.choice)।
3. Demo/preview link-এ (?condition=3 ইত্যাদি) নির্দিষ্ট condition/newsletter জোর করা যায়, এবং এমন session automatically study_phase=preview হিসেবে চিহ্নিত হয় (তাই এটি balance count-কে প্রভাবিত করে না)।

এই পদ্ধতিতে enrollment বাড়ার সাথে সাথে design automatically balanced থাকে — কোনো fixed quota/cap ছাড়াই। STUDY_ENABLED_CONDITIONS environment variable দিয়ে চাইলে শুধু condition ১ ও ২ চালানো যায় (Condition 3 optional/configurable) — জোর-করা preview URL তখনও যেকোনো condition-এর জন্য কাজ করবে।

7 Participant Flow

7.1 ধাপে ধাপে যাত্রা

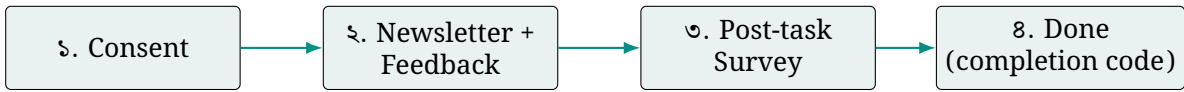


Figure 3: Participant UI-তে চারটি step (`web/src/app/study/[publicId]/page.tsx`)। এটি frontend-এর client-side “step” state; backend-এর `Participant.status` আলাদা (§ 4.4.1)।

Step	API Call	যা ঘটে
0	<code>POST/api/session/start/</code>	BeginButton URL-এর query parameter (source, ref, condition, newsletter) পড়ে এই call করে। Backend <code>choose_cell()</code> দিয়ে (condition, newsletter) ঠিক করে নতুন Participant তৈরি করে, তারপর <code>/study/<public_id></code> -এ redirect করে।
1	<code>POST/api/session/<public_id>/consent/</code>	Consent text দেখানোর পর “I agree” চাপলে <code>consented_at</code> ও <code>status=consented</code> set হয় (idempotent)।
2a	<code>POST/api/session/<public_id>/feedback/initial/</code>	Newsletter ও feedback-form একই page-এ (POPROX-এর নিজস্ব end-of-newsletter feedback block-এর মতো)। Condition 1/2-এ এখানেই <code>final_text</code> set হয়ে যায়; Condition 3-এ প্রথম assistant turn শুরু হয়।
2b	<code>POST.../feedback/chat/, .../feedback/final-draft/</code>	শুধু Condition 3 — § 9 দেখুন।
2c	<code>POST/api/session/<public_id>/feedback/final/</code>	<code>final_text</code> , <code>time_on_task_seconds</code> , <code>revision_count</code> save হয়।
3	<code>POST/api/session/<public_id>/survey/</code>	8-item survey submit; <code>status=completed</code> , <code>completed_at</code> set হয়।
4	—	Done screen-এ <code>public_id</code> completion code হিসেবে দেখানো হয় (Prolific-compatible)।

8 তিনটি Feedback-Elicitation Condition

Copy (participant-facing text) backend-এ রাখা হয়েছে (apps/study/content.py), frontend-এ নয় — যাতে wording pilot-এর পর code না ছুঁয়ে ঠিক করা যায়। সব condition-এ একই core prompt: “How would you like this newsletter to work better for you?” — ইচ্ছাকৃতভাবে neutral, শুধু topic-centric নয়।

Condition	Participant যা দেখেন	Design rationale
1. Just Ask	শুধু core prompt + একটি short explanation, কোনো example/support নেই	Baseline — minimal guidance-এ মানুষ naturally কী লেখেন তা measure করা (RQ1)
2. Examples + Instructions	Prompt + একটি neutral instruction + multiple dimension-এর guiding question (topic, tone, scope, source, length ইত্যাদি) + একটি example	Static support কতটা সাহায্য করে তা measure করা, কিন্তু কোনো নির্দিষ্ট topic/opinion-এর দিকে না ঠেলে
3. Interactive Assistant	Prompt + একটি LLM assistant-এর সাথে ছোট conversation যা clarifying question/suggestion দেয়, কখনো লিখে দেয় না	Interactive support static-এর চেয়ে বেশি সাহায্য করে কিনা তা measure করা (§9)

Condition 2-এর example/guidance ইচ্ছাকৃতভাবে multi-dimensional — শুধু “topic” নয়, বরং tone, locality, person/organization, depth, source type, section-mix, recency, length, diversity, ও context/background — যাতে participant-রা কোনো নির্দিষ্ট দিকে না ঠেলে যান।

9 Condition 3: Interactive Feedback Assistant

এটি codebase-এর সবচেয়ে জটিল অংশ (apps/study/assistant.py, ৩৫১ লাইন)।

9.1 Strict Rules

System prompt-এ স্পষ্টভাবে লেখা — assistant কখনো:

- Participant-এর কথা rewrite/polish করবে না;
- নতুন কোনো preference, topic, entity, বা source invent/suggest করবে না;
- কোনো নির্দিষ্ট topic/opinion/source-এর দিকে participant-কে lead করবে না;
- Sensitive/personal information পেলে neutrally সতর্ক করবে, কোনো judgement ছাড়া।

এর বদলে সে শুধু ১-২টি short, concrete suggestion দেয় বা একটি clarifying question করে — এবং feedback ইতিমধ্যে actionable হলে তা paraphrase করে confirm করে।

9.2 System Prompt কোথায় থাকে

Live prompt থাকে backend/system_prompt.txt-এ (advisor-supplied, POPROX-specific), যা Docker image-এ bake করা থাকে — অর্থাৎ এই file পরিবর্তন করলে backend image পুনরায় build করতে হয় (docker compose up -d --build backend)। File না পাওয়া গেলে assistant.py-তে একটি embedded fallback prompt আছে।

9.3 Conversation Flow

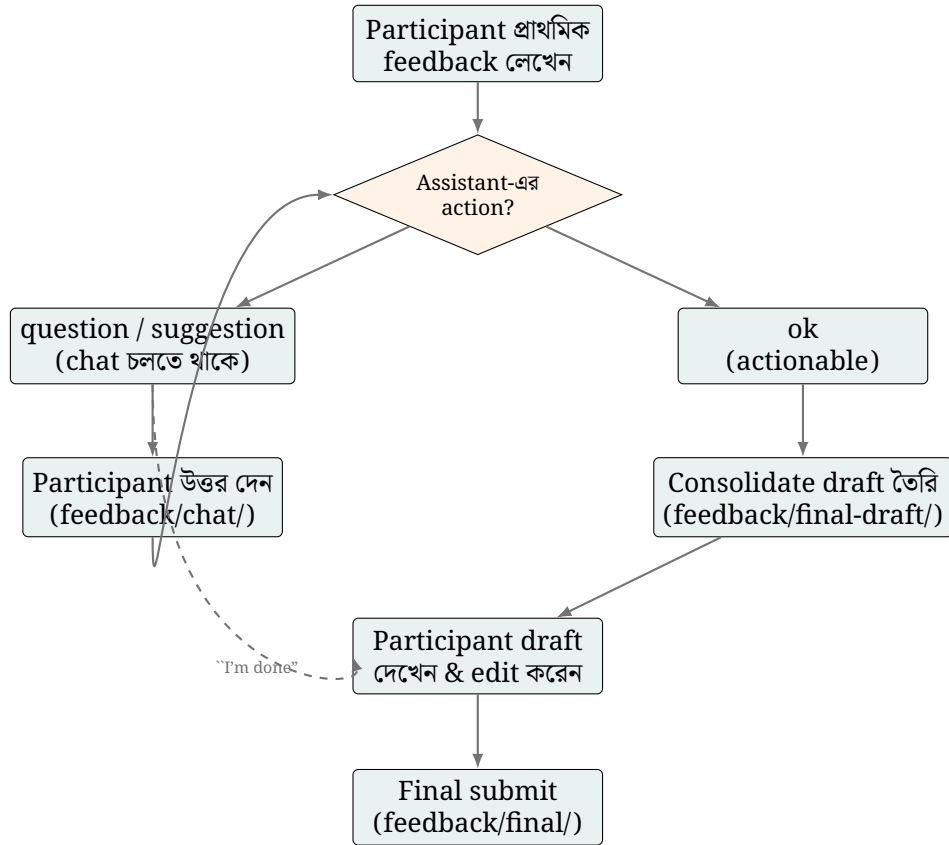


Figure 4: Condition-3 conversation loop। Participant চাইলে যেকোনো সময় “I’m done” চেপে সরাসরি review-panel-এ যেতে পারেন (dashed arrow)।

প্রতিটি turn-এ assistant শুধুমাত্র একটি JSON object return করে:

```
{"action": "suggestion" | "question" | "ok", "message": "..."}

```

CONVERSATION_ADDENDUM মূল prompt-এর সাথে জোড়া লাগে যাতে এটি multi-turn conversation-এ মানিয়ে নেয় (একই question দুবার না করা, একটি turn-এ সর্বোচ্চ একটি question)।

9.4 Newsletter Grounding (read-only context)

Assistant naturally শুধু conversation দেখে — তাই “প্রথম গল্পটা” বা “যুদ্ধের কভারেজ” বললে বুঝতে পারবে না তা কী বোঝাচ্ছে। এই সমস্যা সমাধানে `render_newsletter_context()` participant যে নির্দিষ্ট newsletter পড়েছেন তা system prompt-এর সাথে জুড়ে দেয় — কিন্তু কঠোরভাবে “শুধু reference, suggestion-এর উৎস নয়” হিসেবে চিহ্নিত করে একটি anti-leading guardrail সহ, যাতে assistant newsletter থেকে এমন কিছু suggest না করে যা participant নিজে তোলেননি (এটি study-কে bias করত)।

9.5 Local-only Model Client — কোনো Fallback নেই

`assistant.py` একটি self-hosted, OpenAI-compatible endpoint-এ (`LOCAL_LLM_BASE_URL`, default model `openai/gpt-oss-120b`) call করে — বাইরের কোনো provider import করা নেই, এবং কোনো rule-based fallback-ও নেই। Call fail হলে (endpoint unreachable, timeout, বা parse করতে না পারা JSON) `AssistantUnavailable` exception raise হয়, যা view-তে HTTP 503 হিসেবে ফিরে যায়। Frontend

তখন participant-কে জানায় assistant এই মুহুর্তে পাওয়া যাচ্ছে না, এবং participant ততক্ষণে যা লিখেছেন তা নিয়েই submit করতে পারেন — কোনো canned/deterministic reply দিয়ে চুপচাপ চালিয়ে দেওয়া হয় না।

কেন external LLM provider ব্যবহার করা হয়নি: participant-এর raw feedback (কখনো sensitive information সহ) third-party API-তে না পাঠাতে সচেতনভাবে শুধুমাত্র self-hosted local model ব্যবহার করা হয়েছে — এটি IRB-approved privacy protection-এর সাথে consistent। Fail হলে data বাইরে না পাঠিয়ে সরাসরি 503 দেওয়া হয়, কোনো বিকল্প generated text দিয়ে ভরাট করা হয় না।

9.6 Final Consolidation (final_draft বনাম final_text)

যখন assistant "ok" বলে বা participant "I'm done" চাপেন, frontend `POST.../feedback/final-draft/` call করে। Backend-এর `compose_final_feedback()` পুরো conversation-কে একটি single, first-person statement-এ consolidate করে — এটি একই SYSTEM_PROMPT-এর উপর ভিত্তি করে চলে, শুধু FINALIZE_PROMPT output-format ও task পাল্টে দেয় (JSON turn-reply-এর বদলে একটি plain paragraph), বাকি সব rule (neutrality, কিছু invent না করা) অক্ষুণ্ণ থাকে। এই draft final_draft-এ save হয় এবং participant-কে editable textbox-এ দেখানো হয়।

Participant এই draft স্বাধীনভাবে edit করতে পারেন — যা শেষ পর্যন্ত submit হয় তা final_text-এ যায়। final_draft ও final_text ইচ্ছাকৃতভাবে আলাদা column-এ রাখা হয়েছে, যাতে researcher-রা measure করতে পারেন participant-রা LLM-এর তৈরি draft কতটা faithfully রাখলেন বা কতটা পরিবর্তন করলেন (faithfulness/edit-distance analysis)।

Consolidation বনাম rewriting-এর পার্থক্য: Strict rules বলে assistant কখনো participant-এর কথা rewrite করবে না — তাহলে consolidation step কি rule ভাঙছে? না, কারণ এটি structurally আলাদা একটি step: এটি conversation চলাকালীন ঘটে না (তাই participant-কে influence করতে পারে না), শুধুমাত্র তাঁর নিজের কথাগুলো একত্র/সাজিয়ে দেখায়, এবং সবচেয়ে গুরুত্বপূর্ণ — participant এটি সবসময় দেখেন, edit করতে পারেন, এবং explicitly confirm না করলে submit হয় না।

9.7 Actionability Rubric

Human rater-রা `/researcher/rate-` এ প্রতিটি final feedback একই ৫-dimension rubric-এ score করেন (§ 10)। আগে একটি LLM auto-rater (একই rubric-এ final_text স্বয়ংক্রিয়ভাবে score করত) codebase-এ ছিল, কিন্তু সেটি এবং সংশ্লিষ্ট background job queue & statistics pipeline সরিয়ে ফেলা হয়েছে — এখন rating সম্পূর্ণভাবে human, এবং analysis CSV export-এর পর externally (R/Python-এ) হয়।

Dimension (0/1/2)	কী measure করে
target_specificity	একটি concrete target বলা হয়েছে কি (topic, entity, event, geography, source, section)
direction_operation	স্পষ্ট operation বলা হয়েছে কি (include, exclude, prioritize, diversify, reduce, reorder)
collection_allocation	সামগ্রিক mix, variety, repetition নিয়ে বলা হয়েছে কি
context_persistence	কখন applicable, কতদিন persist করে, কোন exception আছে তা বলা হয়েছে কি
system_feasibility	POPPOX-এর available AP article-pool ও supported operation দিয়ে satisfy করা যায় কিনা

প্রতিটি dimension 0 (absent/unusable), 1 (partial/implied), বা 2 (clear ও directly usable) — total 0–10। সাথে target_level (article, section, collection, unclear) ও একটি short notes।

10 Researcher Dashboard

সব page `/researcher/*`-এর নিচে, `ResearcherShell` component দিয়ে wrap করা (JWT-guard, role-based navigation, CSV export button)।

Page	Access	কী করে
<code>/researcher</code> (Overview)	Manager	Enrollment count (total, completed, final-response, rated), condition-wise table, ও (condition × newsletter) balance-grid
<code>/researcher/rate</code> (Blind rating)	Rater Manager	+ এক-এক করে blind queue থেকে response দেখায় (শুধু <code>final_text</code>); ৫-dimension 0, 1, 2 button, <code>target_level</code> chip, notes, Save/Skip। Save-এর পর queue থেকে সরে যায়।
<code>/researcher/raters</code> (Raters)	Manager	নতুন rater তৈরি (username, email, temporary password, generator সহ), rating-count, last-login, active-status table, password reset বা deactivate; নতুন credential একবারই দেখানো হয়

একাধিক human rater management: মূল researcher account (study manager) `/researcher/raters` থেকে যতজন ইচ্ছা human rater তৈরি করতে পারেন — প্রতিটি rater-এর নিজস্ব login, কখনো account share করা উচিত না (কারণ প্রতিটি score সেই rater-identity-র সাথে জড়িয়ে থাকে, যা পরে externally inter-rater reliability হিসাব করতে কাজে লাগে)।

11 Statistical Analysis

আগে `apps/study/statistics.py`-তে একটি in-app RQ1–RQ3 statistics pipeline ছিল (OLS regression with HC3 robust SE, Holm-corrected planned contrast, Hedges' g effect size, ordinal logistic regression, human↔LLM agreement/ICC/kappa ইত্যাদি) — এই ফাইল ও এর সাথে জড়িত dashboard/API সম্পূর্ণভাবে সরিয়ে ফেলা হয়েছে।

বর্তমান অবস্থা: statistical analysis এই application-এর অংশ নয়। `/api/research/export.csv` থেকে (condition, newsletter, phase, initial/final/draft text, survey score, human-rating mean, সম্পূর্ণ chat_log সহ) সম্পূর্ণ CSV নামিয়ে analysis externally R/Python-এ করা হয়।

12 API Reference

সব endpoint `/api/` দিয়ে শুরু। Interactive documentation: `/api/docs/` (Swagger) ও `/api/redoc/` (drf-spectacular দিয়ে automatically generated, তাই code-এর সাথে সবসময় sync থাকে)।

Participant (public, কোনো login লাগে না)

Method	Path	Description
POST	<code>/api/session/start/</code>	নতুন session (balanced/forced assignment)
GET	<code>/api/session/<public_id>/</code>	Session + newsletter + condition-copy
POST	<code>/api/session/<public_id>/consent/</code>	Consent record
POST	<code>/api/session/<public_id>/feedback/initial/</code>	প্রাথমিক feedback (+ C3 হলে প্রথম assistant turn)
POST	<code>/api/session/<public_id>/feedback/chat/</code>	C3: পরবর্তী conversation-turn
POST	<code>/api/session/<public_id>/feedback/final-draft/</code>	C3: consolidated draft তৈরি
POST	<code>/api/session/<public_id>/feedback/final/</code>	Final feedback submit
POST	<code>/api/session/<public_id>/survey/</code>	Post-task survey submit; status=completed

Authentication

Method	Path	Description
POST	<code>/api/auth/token/</code>	Login (JWT access + refresh)
POST	<code>/api/auth/token/refresh/</code>	Access token refresh
GET	<code>/api/auth/me/</code>	বর্তমান user-এর তথ্য ও role
GET/POST	<code>/api/auth/raters/</code>	Rater list ও তৈরি (manager only)
PATCH	<code>/api/auth/raters/<id>/</code>	Rater update, deactivate, বা password reset (manager only)

Researcher Dashboard

Method	Path	Description
GET	<code>/api/research/overview/</code>	Enrollment summary (manager)
GET	<code>/api/research/responses/</code>	Response list; ?condition=, ?newsletter=, ?phase=, ?unrated=1
GET	<code>/api/research/responses/<id>/</code>	একক response detail (manager)

Method	Path	Description
POST	<code>/api/research/responses/<id>/ratings/</code>	নতুন human rating submit; duplicate হলে 409
GET	<code>/api/research/export.csv</code>	সম্পূর্ণ CSV export (manager)

অন্যান্য: `/api/health/` (health check, AllowAny), `/admin/` (Django admin), `/api/schema/` (OpenAPI JSON schema)।

13 Frontend Architecture

13.1 Folder Organization

Next.js App Router ব্যবহার করা হয়েছে (`web/src/app/`) — প্রতিটি folder একটি route।

Path	কী
<code>app/page.tsx</code>	Homepage — short description + “Begin the study” button
<code>app/study/[publicId]/page.tsx</code>	মূল participant wizard (consent, তারপর newsletter+feedback, তারপর survey, তারপর done)
<code>app/researcher/*</code>	Login + ৩টি dashboard page: overview, rate, raters (§10)
<code>components/shell/</code>	StudyHeader, ResearcherShell (navigation + auth-guard)
<code>components/study/</code>	BeginButton, NewsletterView, FeedbackStep, SurveyStep
<code>components/ui/</code>	Button, Card, Badge, Likert, Spinner, Textarea — ছোট, reusable UI primitive
<code>lib/api.ts</code>	একক typed fetch-client; JWT auto-refresh logic এখানেই
<code>lib/auth.ts</code>	localStorage-এ token রাখা ও পড়া
<code>lib/types.ts</code>	Backend serializer-এর সাথে হুবহু মিলিয়ে রাখা TypeScript interface

13.2 Key Component: NewsletterView

NewsletterView POPROX-এর visual language হুবহু code করে reproduce করে — teal (#008F83) masthead, navy (#003C5E) section-bar, প্রতিটি article-এর thumbnail, label, serif headline, summary। Image load fail হলে (onError) একটি label-সহ placeholder দেখায়, যাতে broken image participant-এর experience নষ্ট না করে।

13.3 Key Component: FeedbackStep

এই component-টি (৩৬৮ লাইন) condition ১, ২, ৩ — তিনটিরই UI single file-এ ধারণ করে, একটি ছোট state machine দিয়ে:

- `phase: "writing" | "chat"` — প্রথমে সবাই লেখেন; শুধু Condition 3-এ "chat" phase চালু হয় (যদি প্রথম assistant-action "none" না হয়)।
- `finalDirtyRef` (একটি `useRef`) — participant একবার draft হাতে edit করলে, নতুন chat-turn-এ আর automatically draft refresh/overwrite হয় না — তাঁদের edit respect করা হয়।
- Optimistic UI — `handleSend()`-এ user-এর message তাৎক্ষণিকভাবে chat-এ দেখানো হয় (server response-এর জন্য wait না করে); fail হলে rollback করে draft ফিরিয়ে দেয়।
- `seconds()` helper `time_on_task_seconds` measure করে।
- Enter-key chat-এ message পাঠায় (Shift+Enter নতুন line) — চ্যাট-অ্যাপের প্রচলিত UX pattern।

13.4 API Client: JWT Auto-refresh

`lib/api.ts`-এর `request()` function প্রতিটি call-এ ৪০১ পেলে (এবং আগে `retry` না করা থাকলে) automatically `refreshAccessToken()` call করে, নতুন access token দিয়ে একবার আবার চেষ্টা করে। একাধিক parallel request একই সময়ে ৪০১ পেলে, একটিমাত্র refresh-request share হয় (`refreshRequest` promise variable দিয়ে)।

14 Deployment ও Operations

14.1 Docker Compose Services

Service	Image/Command	Note
db	postgres:16	Loopback-only port (127.0.0.1:5436), pg_isready healthcheck
backend	backend/Dockerfile (Django, Gunicorn)	network_mode: host; startup-এ migrate + seed_study automatically চলে
web	web/Dockerfile (multi-stage: builder + runner)	NEXT_PUBLIC_API_BASE_URL build-time-এ bake হয় (Next.js-এর নিয়ম)

গুরুত্বপূর্ণ: backend ও web দুটোরই Dockerfile source code image-এ copy করে — কোনো volume mount নেই। তাই code পরিবর্তনের পর শুধু container restart করলে হবে না; image পুনরায় build করতে হয়: `docker compose up -d --build backend (বা web)`। এটি না করলে stale code চলতেই থাকবে।

14.2 run.sh — Convenience Wrapper Script

Command	কাজ
<code>./run.sh up</code>	Build + সম্পূর্ণ stack background-এ চালু
<code>./run.sh down</code> <code>restart</code>	/ বন্ধ/পুনরায় চালু (database volume অক্ষত থাকে)
<code>./run.sh reset</code>	Database volume মুছে নতুন করে শুরু (re-seed)
<code>./run.sh rebuild</code>	শূন্য থেকে image পুনর্নির্মাণ
<code>./run.sh logs</code> <code>[svc], ps</code>	Log/status দেখা
<code>./run.sh seed</code> <code>migrate / superuser</code>	/ Seed/migrate/superuser
<code>./run.sh manage</code> <code><args> / shell</code>	Backend container-এ <code>manage.py</code> বা shell

14.3 Environment Variables

Name	Where	কাজ
DATABASE_URL	backend	PostgreSQL DSN
DJANGO_SECRET_KEY, DJANGO_DEBUG, DJANGO_ALLOWED_HOSTS	backend	Core Django settings

Name	Where	কাজ
CORS_ALLOWED_ORIGINS, CORS_ALLOW_ALL_ORIGINS, CSRF_TRUSTED_ORIGINS	backend	Browser-security
RESEARCHER_USERNAME, RESEARCHER_PASSWORD	backend	Seeded dashboard login
LOCAL_LLM_BASE_URL, LOCAL_LLM_MODEL, LOCAL_LLM_API_KEY, LOCAL_LLM_TIMEOUT_SECONDS	backend	Condition-3 assistant (§ 9)
STUDY_ENABLED_CONDITIONS	backend	Balanced assignment-এ active condition (§ 6)
STUDY_PHASE	backend	pilot (default) বা main (§ 4.4.2)
BACKEND_PORT, WEB_PORT, DB_PORT	compose	Port mapping
NEXT_PUBLIC_API_BASE_URL	web (build-time)	Browser থেকে backend-এর URL
POSTGRES_DB, POSTGRES_USER, POSTGRES_PASSWORD	compose	Database credential

14.4 Port Map

Default: web 3000, backend 8001, DB 5436 (loopback-only)। এই server-এ (অন্যান্য project 3000/3001 ব্যবহার করায়) .env-এ WEB_PORT=3002 set করা — তাই এই server-এ participant site <http://localhost:3002>। Backend container network_mode: host ব্যবহার করে যাতে host-এর 127.0.0.1:8123-এ চলা local LLM-এ পৌঁছাতে পারে। Remote থেকে demo দেখাতে হলে দুটো port-ই SSH দিয়ে forward করতে হয়।

15 Testing ও Quality

- Backend: `pytest` (`backend/tests/`) — auth, assistant logic, balanced assignment, full flow, permission cover করে। Production image-এ `pytest` থাকে না (light রাখতে); run করতে container-এ `pip install pytest pytest-django` লাগে।
- Frontend: `npm run test` (Vitest), `npm run lint` (ESLint, next/core-web-vitals), `npm run build` (production build + TypeScript type-check)।

16 Security ও Privacy Considerations

- কোনো PII সংগ্রহের design নেই — consent-text-এ স্পষ্টভাবে request করা হয় নাম, ঠিকানা, ইমেইল, ফোন না লিখতে; assistant sensitive information দেখলে neutrally সতর্ক করে (§ 9)।
- Data তৃতীয়-পক্ষের কাছে যায় না — LLM call শুধু self-hosted local endpoint-এ যায়, কোনো বাইরের API (OpenAI, Google ইত্যাদি) নেই।
- JWT স্বল্পস্থায়ী — access token ৮ ঘণ্টা, refresh ৭ দিন; সব researcher-endpoint IsAuthenticated (default) + IsResearcher বা IsStudyManager দিয়ে সুরক্ষিত।
- CORS/CSRF স্পষ্টভাবে allowlist-ভিত্তিক production-এ (CORS_ALLOW_ALL_ORIGINS default বন্ধ)।
- Rating overwrite করা যায় না — একজন rater একটি response একবারই score করতে পারেন (unique(feedback, rater)); ভুল score ঠিক করতে database-level access লাগে, accidental double-submit থেকে data protect থাকে (§ 4)।
- Secret environment variable-এ — .env .gitignore-এ (repo-তে শুধু .env.example); default password production-এ অবশ্যই পাল্টাতে হবে।

17 সম্ভাব্য প্রশ্ন ও উত্তর

প্রশ্ন 1. ব্যাকএন্ডে Django + DRF কেন বেছে নেওয়া হলো, অন্য কিছু না?

উত্তর: Research study-তে দ্রুত ও নির্ভরযোগ্য data-modeling জরুরি — Django-র ORM, built-in admin panel (data দ্রুত inspect করার জন্য), migration system, এবং DRF-এর serializer/permission framework এই ধরনের CRUD-heavy, permission-sensitive application-এর জন্য proven ও দ্রুত development দেয়।

প্রশ্ন 2. কেন PostgreSQL, আর JSONField কেন দরকার হলো?

উত্তর: chat_log ও sections-এর structure nested ও variable (প্রতিটি conversation-এ ভিন্ন সংখ্যক turn) — এর জন্য প্রতিটি turn-এর জন্য আলাদা table বানানো unnecessary complexity যোগ করত। PostgreSQL-এর native JSONField এখানে উপযুক্ত।

প্রশ্ন 3. Participant-দের কীভাবে identify করা হয়? তাদের কি account লাগে?

উত্তর: না। প্রতিটি session-এর একটি random UUID (public_id) — এটিই তাঁদের URL এবং সবশেষে completion code। কোনো password/login লাগে না, যা research-এ participation friction কমায়।

প্রশ্ন 4. Condition 3-এ assistant কীভাবে participant-এর কথা বদলে দেওয়া থেকে বিরত থাকে?

উত্তর: তিনটি স্তরে — (ক) system prompt-এ স্পষ্ট strict rules (rewrite, invent, lead করবে না); (খ) structurally chat ও final consolidation আলাদা step; (গ) participant সবসময় final text নিজে দেখেন, edit করতে পারেন, এবং explicitly confirm না করলে কিছুই submit হয় না।

প্রশ্ন 5. Local LLM যদি fail বা down হয়, তখন কী হয়?

উত্তর: কোনো fallback নেই — backend AssistantUnavailable raise করে, যা HTTP 503 হিসেবে ফিরে যায়। Frontend participant-কে জানায় assistant এই মুহূর্তে পাওয়া যাচ্ছে না; participant ততক্ষণে যা লিখেছেন তা নিয়েই submit করতে পারেন।

প্রশ্ন 6. বাইরের LLM provider (OpenAI, Gemini ইত্যাদি) কেন ব্যবহার করা হয়নি?

উত্তর: Privacy রক্ষায় — participant-এর raw, potentially sensitive feedback text কোনো third-party API-তে পাঠানো হয় না। সবকিছু self-hosted, controlled environment-এ থাকে।

প্রশ্ন 7. Balanced assignment কীভাবে কাজ করে?

উত্তর: প্রতিটি নতুন participant (condition, newsletter)-এর সবচেয়ে কম-জনবহুল cell-এ বসেন (tie হলে random বাছাই) — এভাবে ৯টি cell-এর মধ্যে সংখ্যা automatically সমান থাকে।

প্রশ্ন 8. Human rater-দের কেন blind রাখা হয়েছে?

উত্তর: Anchoring bias এড়াতে — condition, LLM-score, বা অন্য rater-এর score জানলে তা rater-এর judgement-কে unknowingly প্রভাবিত করতে পারে।

প্রশ্ন 9. study_phase (preview, pilot, main) দিয়ে কোন সমস্যা সমাধান হয়?

উত্তর: Demo সেশন (?condition=3) যেন বাস্তব ডেটার সাথে মিশে confirmatory result দৃষিত না করে — এগুলো automatically preview phase-এ যায়, যা default-ভাবে analysis ও balancing থেকে বাদ থাকে।

প্রশ্ন 10. কেন Docker Compose-এ backend-এর network_mode: host ব্যবহার করা হয়েছে?

উত্তর: এই Linux server-এ backend container-এর host machine-এ চলা local LLM (127.0.0.1:8123) পর্যন্ত পৌঁছাতে default Docker bridge network-এ hairpin সমস্যা হয়; host networking সরাসরি host-এর network stack ব্যবহার করে এই সমস্যা এড়ায়।

প্রশ্ন 11. researcher ও rater account-এর পার্থক্য backend-এ কীভাবে enforce করা হয়?

উত্তর: Django Group (human_raters) দিয়ে। IsStudyManager permission class is_staff=True এবং সেই group-এ না থাকা — দুটো condition একসাথে check করে।

প্রশ্ন 12. CSV export-এ কী কী থাকে?

উত্তর: প্রতিটি participant-এর প্রতি একটি row — condition, newsletter, phase, initial/final/draft

text, assistant action/turn count, survey-score, per-dimension human-rating mean, এবং সম্পূর্ণ chat_log JSON হিসেবে (apps/study/views.py-এর ExportView)।

প্রশ্ন 13. ভবিষ্যতে নতুন condition বা newsletter যোগ করা কতটা সহজ?

উত্তর: নতুন newsletter শুধু seed_data/newsletters.json-এ একটি entry যোগ করে seed_study চালালেই যোগ হয়ে যায় (code পরিবর্তন লাগে না)। নতুন condition যোগ করতে Condition enum, content.py-তে copy, এবং FeedbackStep-এ branch যোগ করতে হবে — moderate পরিবর্তন, কিন্তু architecture তা support করে।

18 Glossary

Term	সংক্ষিপ্ত ব্যাখ্যা
API / Endpoint	নির্দিষ্ট URL যেখানে frontend backend-কে request পাঠায়
JWT	JSON Web Token — login-এর পর issue করা signed token, প্রতিটি request-এ identity প্রমাণ করে
ORM	Object-Relational Mapping — Django-র model class-কে SQL table-এ রূপান্তরকারী layer
Migration	Model পরিবর্তনের history, যা database schema ধাপে ধাপে update করে
Serializer (DRF)	Django model ↔ JSON রূপান্তরকারী class, validation সহ
Fixture / Seed	প্রাথমিক data (এখানে newsletter + researcher account) automatically load করা
Rubric	একটি structured scoring standard (এখানে ৫-dimension actionability rubric)
CORS / CSRF	Browser-security mechanism — কোন origin থেকে API call/form-submission গ্রহণযোগ্য তা নিয়ন্ত্রণ করে
Blind rating	Rater-কে condition/score গোপন রেখে evaluate করানো, bias এড়াতে

— End of Document —